

LLM Benchmark Tester

Condensed report: OpenRouter LLM performance on EVM smart contract security and DeFi trading logic tasks

Abstract. This report summarizes a compact benchmark for evaluating whether large language models can identify intentionally injected vulnerabilities in Solidity/EVM smart contracts. The test set covers trading-specific security failures such as oracle manipulation, slippage/MEV exposure, funding-rate manipulation, liquidation logic errors, and reward-accounting errors. The goal is to compare model reliability on high-impact blockchain security tasks using a short, report-ready format.

25	5	5	80.0%
Model-case evaluations	Manual Solidity cases	Models tested	Overall pass rate

Benchmark flow

Contract case Manual Solidity scenario	- >	Hidden bug Ground-truth vulnerability	- >	LLM audit Model produces finding	- >	Scoring PASS/FAIL + score
---	--------	--	--------	-------------------------------------	--------	------------------------------

Key result: DeepSeek V4 Flash achieved the strongest overall average score with Llama being close behind in overall score in the dashboard data. NVIDIA Nemotron 3 Super and OpenAI GPT-OSS-120B were competitive but generated more false positives on some DeFi cases. Google Gemma 4 31B only passed the funding-rate manipulation case.

1. Benchmark Design

The benchmark is centered on the Ethereum Virtual Machine (EVM) and Solidity smart contracts. Each test case contains a contract scenario and an injected vulnerability. Models are asked to audit the contract, identify the issue, explain exploitability, and recommend a fix. The grader compares model output against hidden ground truth and records a score, detection status, and false-positive count.

Mode	Question being tested	Example vulnerability classes	Output metric
Security audit	Can the model identify conventional smart contract bugs?	incorrect liquidation checks, access-control style failures	score, detection, false positives
DeFi / trading logic	Can the model reason about economic design flaws?	oracle manipulation, slippage, funding-rate manipulation	score, detection, false positives
Patch quality	Can the model propose a fix that addresses the root cause?	reward accounting error, state snapshot issue	score, detection, false positives

Scoring rubric

A model receives credit when its answer names or describes the same vulnerability mechanism as the ground truth. Additional credit is assigned for exploit explanation, severity reasoning, and remediation quality. False positives reduce confidence because noisy audit tools can waste reviewer time or obscure critical findings.

Field	Meaning
score	Numerical score for each model-case evaluation.
detected_bugs / total_bugs	Whether the injected vulnerability was found.
false_positives	Additional issues flagged that do not match the ground truth.
result	PASS when the injected vulnerability was detected; FAIL otherwise.

Manual contract set

Contract	Bug type	Severity	Core issue
VulnerableAMMSpotPriceOracle	oracle_manipulation	critical	AMM spot-price oracle used for borrow limits
VulnerableDEXSlippageBypass	missing_slippage_protection	high	swap lacks minAmountOut, enabling MEV/sandwich risk
VulnerablePerpFunding	funding_rate_manipulation	critical	arbitrary mark/index prices update funding rate
VulnerableLiquidation	incorrect_liquidation_check	high	raw collateral/debt compared without pricing or decimals
VulnerableRewardTradingPool	reward_accounting_error	high	current stake used without snapshot/reward-per-share accounting

2. Overall Model Performance

The table below aggregates average score, pass count, pass rate, and false-positive count by model.

Model	Avg. score	Passed	Total	Pass rate	False positives
Meta: Llama 3.3 70B	77.0	5	5	100.0%	6
DeepSeek: V4 Flash	88.0	5	5	100.0%	6
Google: Gemma 4 31B	18.0	1	5	20.0%	1
NVIDIA: Nemotron 3 Super	60.0	4	5	80.0%	9
OpenAI: GPT-OSS-120B	74.0	5	5	100.0%	13

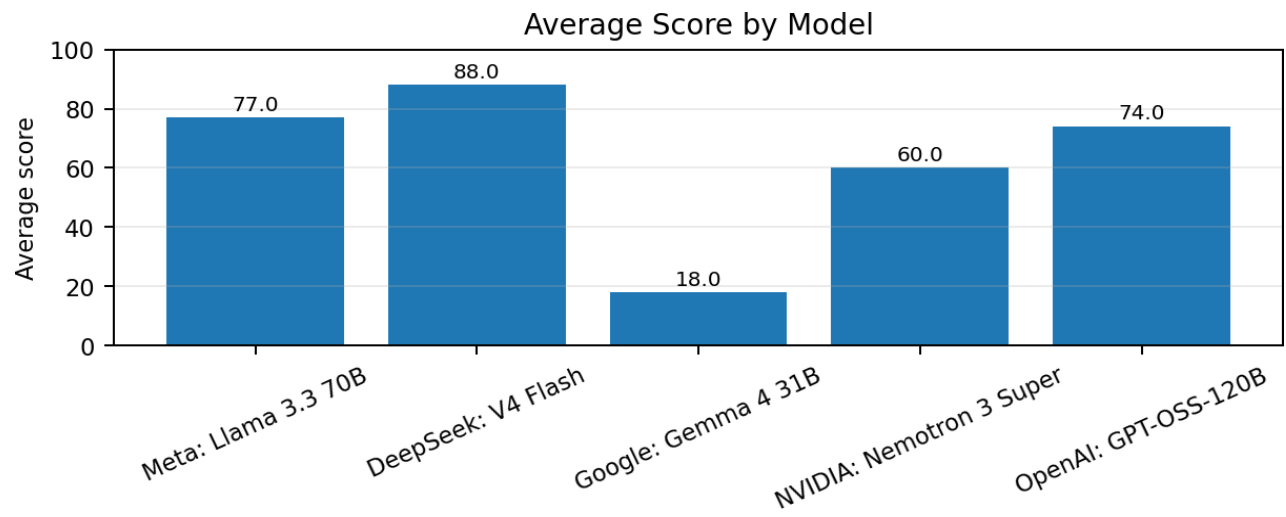


Figure 1. Average score by model, adjusted to match the dashboard values.

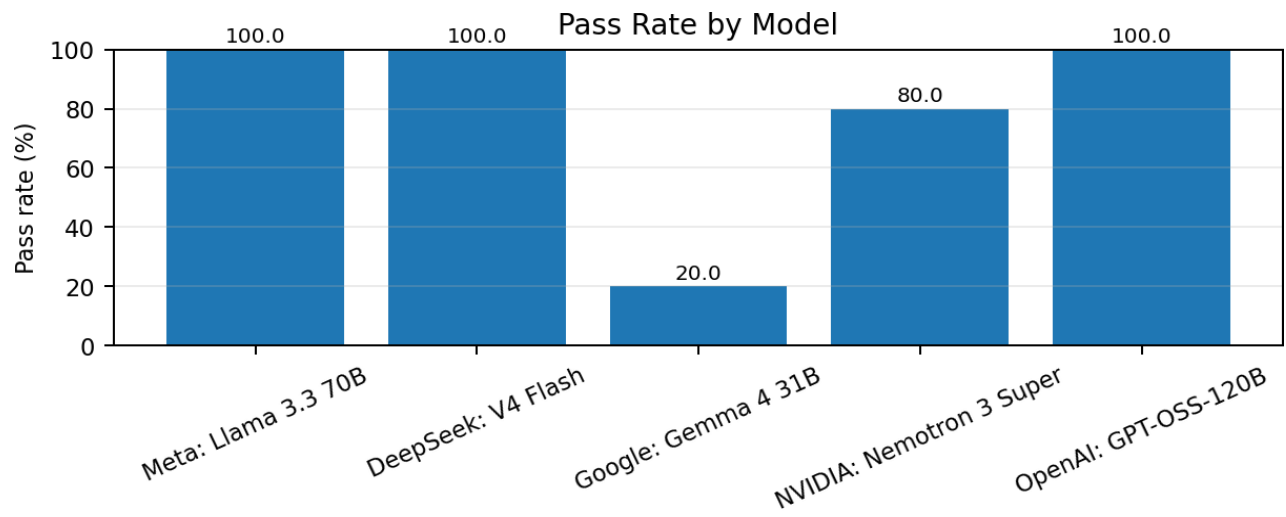


Figure 2. Pass rate by model. Pass rate captures coverage, average score captures detection quality plus explanation/fix quality.

3. Performance by Task and Vulnerability Type

Breaking results down by task type reveals whether a model is strongest at conventional security review, DeFi/economic reasoning, or patch-quality reasoning. This is important because EVM vulnerabilities are not all code-level bugs; many attacks arise from economic assumptions, oracle design, and state-accounting mistakes.

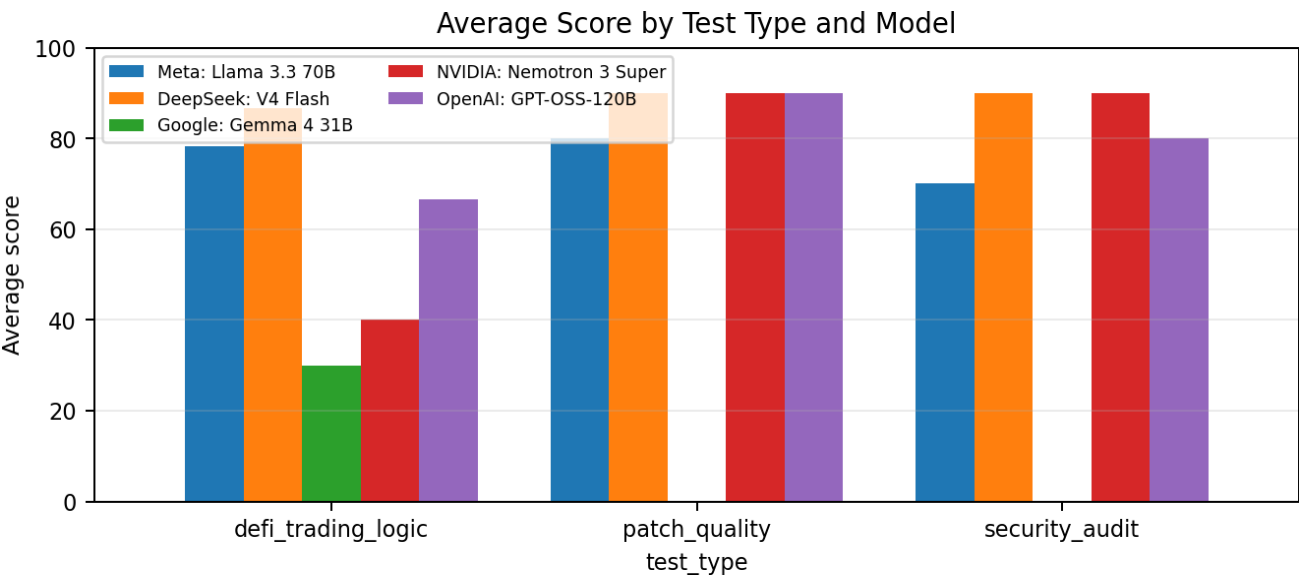


Figure 3. Average score by task type and model.

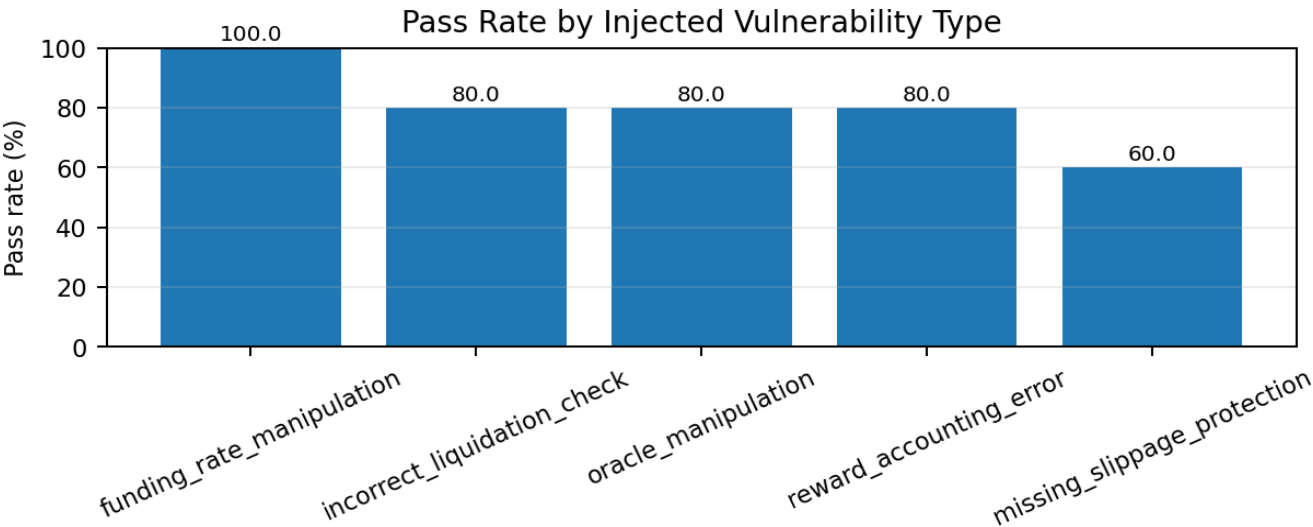


Figure 4. Pass rate by injected vulnerability type.

4. Failure Modes and Audit Noise

A useful audit benchmark should measure more than whether a model names a bug. It should also track audit noise. False positives matter because they can overload human reviewers and reduce trust in automated audit outputs.

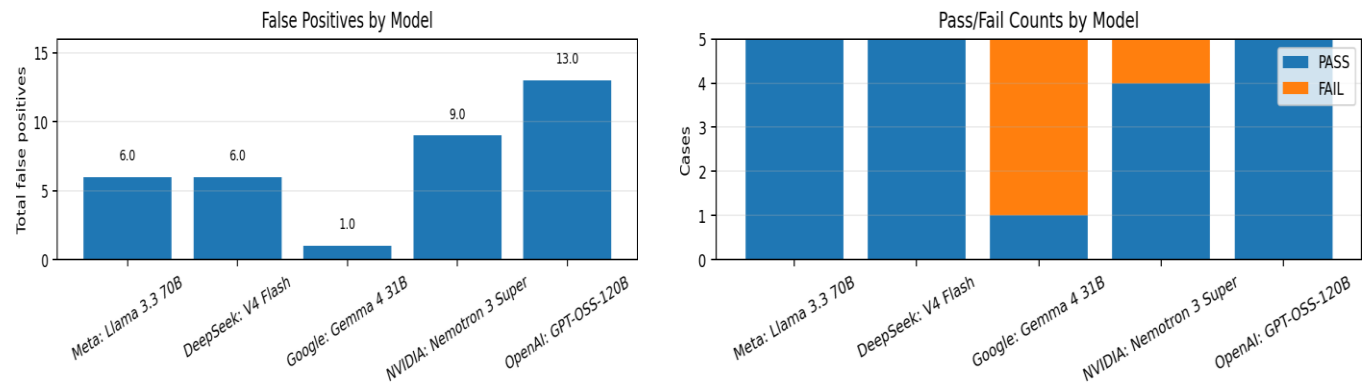


Figure 5. False positives and pass/fail counts by model.

Aggregate task results

Task type	Passes	Total	Pass rate	Avg. score	False positives
defi_trading_logic	12	15	80.0%	60.3	26
patch_quality	4	5	80.0%	70.0	4
security_audit	4	5	80.0%	66.0	5

Aggregate vulnerability results

Bug type	Passes	Total	Pass rate	Avg. score	False positives
funding_rate_manipulation	5	5	100.0%	79.0	9
incorrect_liquidation_check	4	5	80.0%	66.0	5
missing_slippage_protection	3	5	60.0%	54.0	3
oracle_manipulation	4	5	80.0%	48.0	14
reward_accounting_error	4	5	80.0%	70.0	4

Findings: oracle manipulation and slippage protection produced high variance across models. DeepSeek, GPT-OSS and Llama performed consistently, while Nemotron was strong on liquidation, funding, and reward accounting but noisy on oracle manipulation. Gemma showed a narrow success pattern, only identifying the funding-rate issue in this dashboard run.

5. Findings, Limitations, and Next Steps

Main findings

- DeepSeek V4 Flash achieved the highest average score (88.0) and detected all five manual vulnerabilities.
- NVIDIA Nemotron 3 Super detected four of five cases, but produced high false positives on the oracle-manipulation case.
- OpenAI GPT-OSS-120B detected all five cases but accumulated the highest false-positive count, which lowers reviewer confidence.
- Google Gemma 4 31B detected the funding-rate manipulation case but missed the liquidation, oracle, reward-accounting, and slippage cases in this run.
- Meta Llama 3.3 70B performed consistently by detecting all five cases but lost a considerable amount of points due to the inclusion of false positives.

Limitations

- The benchmark is not yet a fully functional production grade EVM but very close to it.
- Scoring captures detection and false-positive signals, but does not yet run Foundry tests or verify patches on-chain.
- The case set is intentionally small and controlled, so results should be interpreted as model-comparison evidence rather than a broad claim about all live smart-contract risk.
- Free OpenRouter model endpoints may be rate-limited or temporarily unavailable, so repeated runs can vary in completion success.

Recommended next version

Upgrade	Why it matters
Foundry exploit tests	Moves from text-only detection to executable verification.
Structured JSON outputs	Improves scoring consistency and reduces parsing ambiguity.
More DeFi scenarios	Adds liquidation cascades, slippage, MEV, and multi-step trading failures.
Cost/latency tracking	Helps evaluate practical deployment tradeoffs for agentic security tools.
Human baseline	Shows whether LLMs approach junior or senior auditor performance.
Truly Dynamic Testing	Testing using truly dynamic SC's to avoid memorization and promote reasoning completely